

Multiclass Support Vector Machine exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the SVM
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
In [64]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [65]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK FIRST ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [66]: # Run some setup code for this notebook.
from __future__ import print_function
import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipy
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:
`%reload_ext autoreload`

CIFAR-10 Data Loading and Preprocessing

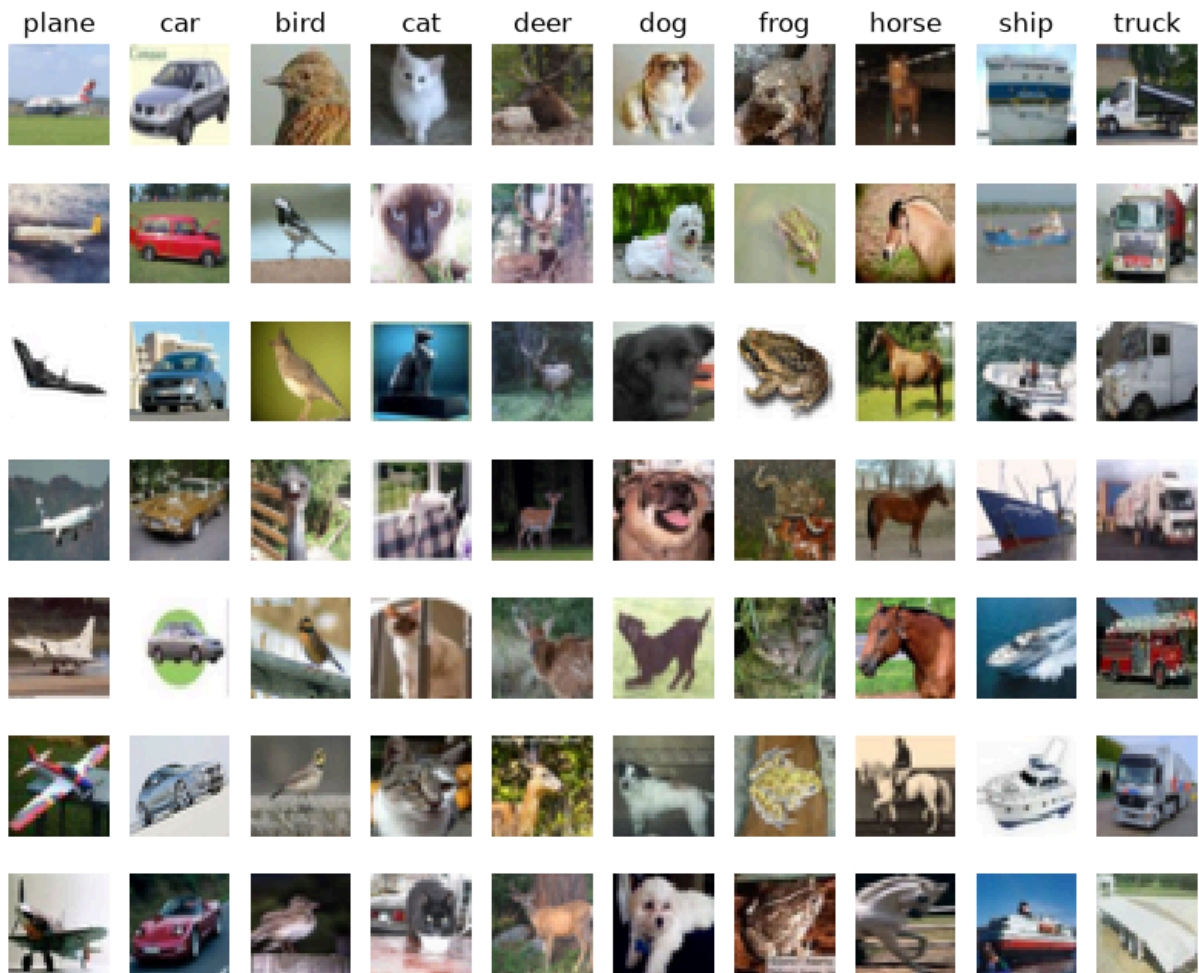
```
In [67]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs682/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may ca
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass
# Don't forget to run get_datasets.sh, or this will throw an error
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Clear previously loaded data.
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape: (10000, 32, 32, 3)
Test labels shape: (10000,)
```

```
In [68]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 's
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
In [69]: # Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]
```

```
# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Train data shape: (49000, 32, 32, 3)
Train labels shape: (49000,)
Validation data shape: (1000, 32, 32, 3)
Validation labels shape: (1000,)
Test data shape: (1000, 32, 32, 3)
Test labels shape: (1000,)
```

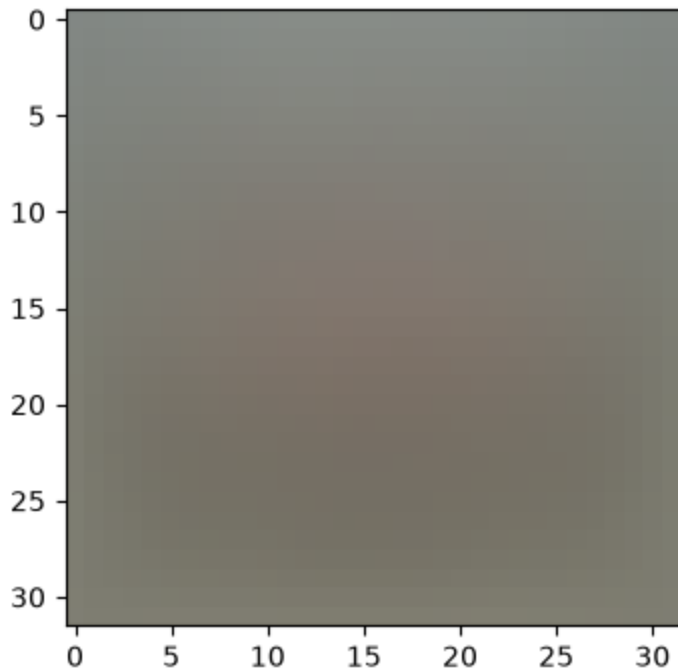
```
In [70]: # Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape: (49000, 3072)
Validation data shape: (1000, 3072)
Test data shape: (1000, 3072)
dev data shape: (500, 3072)
```

```
In [71]: # Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean image
plt.show()
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```



```
In [72]: # second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image
```

```
In [73]: # third: append the bias dimension of ones (i.e. bias trick) so that our SVM
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)

(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

SVM Classifier

Your code for this section will all be written inside **cs682/classifiers/linear_svm.py**.

As you can see, we have prefilled the function `svm_loss_naive` which uses for loops to evaluate the multiclass SVM loss function.

```
In [74]: # Evaluate the naive implementation of the loss we provided for you:
from cs682.classifiers.linear_svm import svm_loss_naive
import time

# generate a random SVM weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001
```

```
loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))
```

loss: 9.149796

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the SVM cost function and implement it inline inside the function `svm_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient correctly, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

```
In [75]: # Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions,
# compare them with your analytically computed gradient. The numbers should
# almost exactly along all dimensions.
from cs682.gradient_check import grad_check_sparse
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = svm_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: 4.597325 analytic: 4.597325, relative error: 2.240111e-11
numerical: -7.762665 analytic: -7.762665, relative error: 1.065460e-11
numerical: 17.781522 analytic: 17.781522, relative error: 1.096751e-11
numerical: 12.290537 analytic: 12.290537, relative error: 2.989880e-11
numerical: -14.311770 analytic: -14.311770, relative error: 7.345401e-12
numerical: -19.019036 analytic: -19.019036, relative error: 4.140373e-12
numerical: -8.931383 analytic: -8.931383, relative error: 3.421086e-11
numerical: 34.412771 analytic: 34.412771, relative error: 8.254929e-13
numerical: 6.382881 analytic: 6.382881, relative error: 2.571820e-11
numerical: -1.213852 analytic: -1.214176, relative error: 1.332281e-04
numerical: 4.588690 analytic: 4.588690, relative error: 1.405596e-10
numerical: 2.754356 analytic: 2.754356, relative error: 1.181884e-11
numerical: -18.333518 analytic: -18.333518, relative error: 1.339114e-11
numerical: -19.898355 analytic: -19.898355, relative error: 1.707586e-12
numerical: -4.172778 analytic: -4.172778, relative error: 3.382207e-11
numerical: -19.779520 analytic: -19.779520, relative error: 9.516133e-12
numerical: 3.882263 analytic: 3.882263, relative error: 3.579372e-11
numerical: 22.230817 analytic: 22.211512, relative error: 4.343833e-04
numerical: -14.674736 analytic: -14.674736, relative error: 8.328328e-12
numerical: 4.813613 analytic: 4.813613, relative error: 4.219777e-11
```

Inline Question 1:

It is possible that once in a while a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a gradient check could fail? How would change the margin affect of the frequency of this happening? *Hint: the SVM loss function is not strictly speaking differentiable*

Answer: With scores $s_j = W_j \cdot x_i$, the hinge loss is

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

and its gradient uses the indicator $1(J) = 1$ if class j violates the margin ($s_j - s_{y_i} + 1 > 0$ and $j \neq y_i$), 0 otherwise:

$$\frac{\partial L_i}{\partial W_j} = 1(J) x_i, \quad \frac{\partial L_i}{\partial W_{y_i}} = - \sum_{j \neq y_i} 1(J) x_i$$

Cause of the mismatch: $\max(0, \cdot)$ has a kink where $s_j - s_{y_i} + 1 = 0$, so the loss is not differentiable there and $1(J)$ jumps from 0 to 1. The numerical gradient uses a centered difference $\frac{L(W+h) - L(W-h)}{2h}$. If a margin is within h of the kink, the two evaluation points fall on opposite sides, so the numerical estimate averages the two slopes while the analytic gradient picks one side.

Is it a concern? No. It is an artifact of checking the gradient numerically at a non-differentiable point, not a bug in the analytic gradient. It affects only an occasional dimension, and gradient descent works fine with this subgradient.

1-D example: $f(x) = \max(0, x)$ at $x = 0$. The analytic gradient we use is 0 (with the indicator convention $1 = 0$ when the margin is not > 0), but the numerical gradient is $\frac{f(h) - f(-h)}{2h} = \frac{h - 0}{2h} = 0.5$.

Effect of the margin: Call the margin Δ (it is 1 in the lecture formula). At initialization W is small, so all $s \approx 0$ and every term is $s_j - s_{y_i} + \Delta \approx \Delta = 1$, far from the kink, so failures are rare. Making Δ smaller (toward 0) puts many terms right at the kink and makes mismatches much more frequent; a larger Δ makes them rarer. In general the frequency depends on how many margin terms land within h of 0.

```
In [76]: # Next implement the function svm_loss_vectorized; for now only compute the
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))
```

```

from cs682.classifiers.linear_svm import svm_loss_vectorized
tic = time.time()
loss_vectorized, _ = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much
print('difference: %f' % (loss_naive - loss_vectorized))

```

Naive loss: 9.149796e+00 computed in 0.056214s
 Vectorized loss: 9.149796e+00 computed in 0.006264s
 difference: -0.000000

```

In [77]: # Complete the implementation of svm_loss_vectorized, and compute the gradient
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)

```

Naive loss and gradient: computed in 0.058421s
 Vectorized loss and gradient: computed in 0.002161s
 difference: 0.000000

Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss.

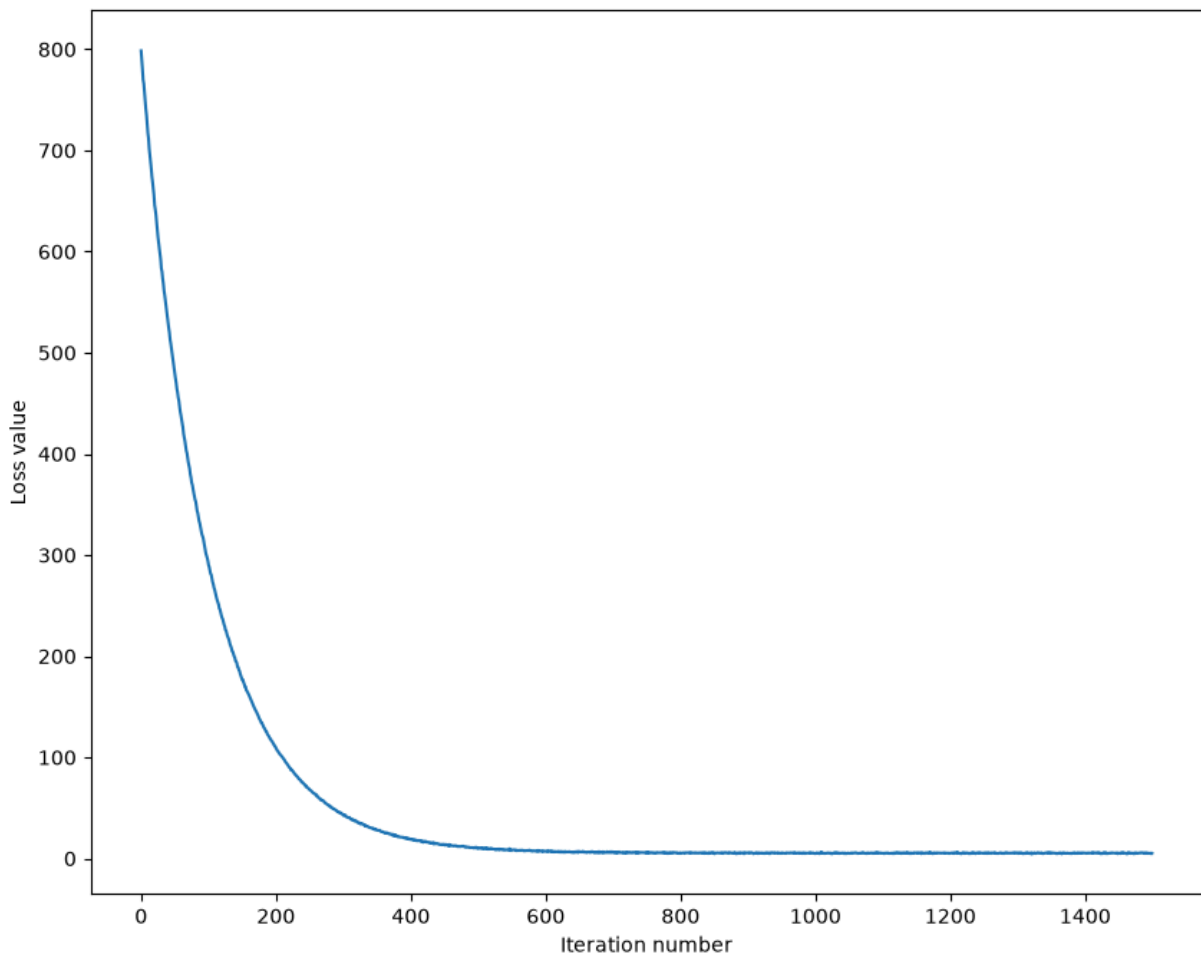
```

In [78]: # In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs682.classifiers import LinearSVM
svm = LinearSVM()
tic = time.time()
loss_hist = svm.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                      num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))

```

```
iteration 0 / 1500: loss 798.548101
iteration 100 / 1500: loss 290.237814
iteration 200 / 1500: loss 109.144227
iteration 300 / 1500: loss 42.514514
iteration 400 / 1500: loss 18.795323
iteration 500 / 1500: loss 9.899851
iteration 600 / 1500: loss 7.320198
iteration 700 / 1500: loss 6.217382
iteration 800 / 1500: loss 4.998459
iteration 900 / 1500: loss 5.601807
iteration 1000 / 1500: loss 4.970029
iteration 1100 / 1500: loss 5.215555
iteration 1200 / 1500: loss 5.384175
iteration 1300 / 1500: loss 5.496114
iteration 1400 / 1500: loss 5.328431
That took 2.376542s
```

```
In [79]: # A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



```
In [80]: # Write the LinearSVM.predict function and evaluate the performance on both
# training and validation set
y_train_pred = svm.predict(X_train)
```

```
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = svm.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

training accuracy: 0.370898
validation accuracy: 0.387000

```
In [81]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able
# to get a classification accuracy of about 0.4 on the validation set.
learning_rates = [1e-7, 5e-5]
regularization_strengths = [2.5e4, 5e4]

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1 # The highest validation accuracy that we have seen so far.
best_svm = None # The LinearSVM object that achieved the highest validation

#####
# TODO:
# Write code that chooses the best hyperparameters by tuning on the validation
# set. For each combination of hyperparameters, train a linear SVM on the
# training set, compute its accuracy on the training and validation sets, and
# store these numbers in the results dictionary. In addition, store the best
# validation accuracy in best_val and the LinearSVM object that achieves this
# accuracy in best_svm.
#
# Hint: You should use a small value for num_iters as you develop your
# validation code so that the SVMs don't take much time to train; once you are
# confident that your validation code works, you should rerun the validation
# code with a larger value for num_iters.
#####
# Extend the provided values with a few more around 1e-7. Pairs with
# lr * reg >= 1 are skipped: the regularization step then overshoots and
# training diverges (this is the case for lr = 5e-5).
search_lrs = learning_rates + [2e-7, 5e-7]
search_regs = regularization_strengths + [1e4]

for lr in search_lrs:
    for reg in search_regs:
        if lr * reg >= 1:
            continue
        svm = LinearSVM()
        svm.train(X_train, y_train, learning_rate=lr, reg=reg,
                  num_iters=1500, verbose=False)
        train_accuracy = np.mean(svm.predict(X_train) == y_train)
        val_accuracy = np.mean(svm.predict(X_val) == y_val)
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_svm = svm
#####
```

```
#
# END OF YOUR CODE
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' % best)

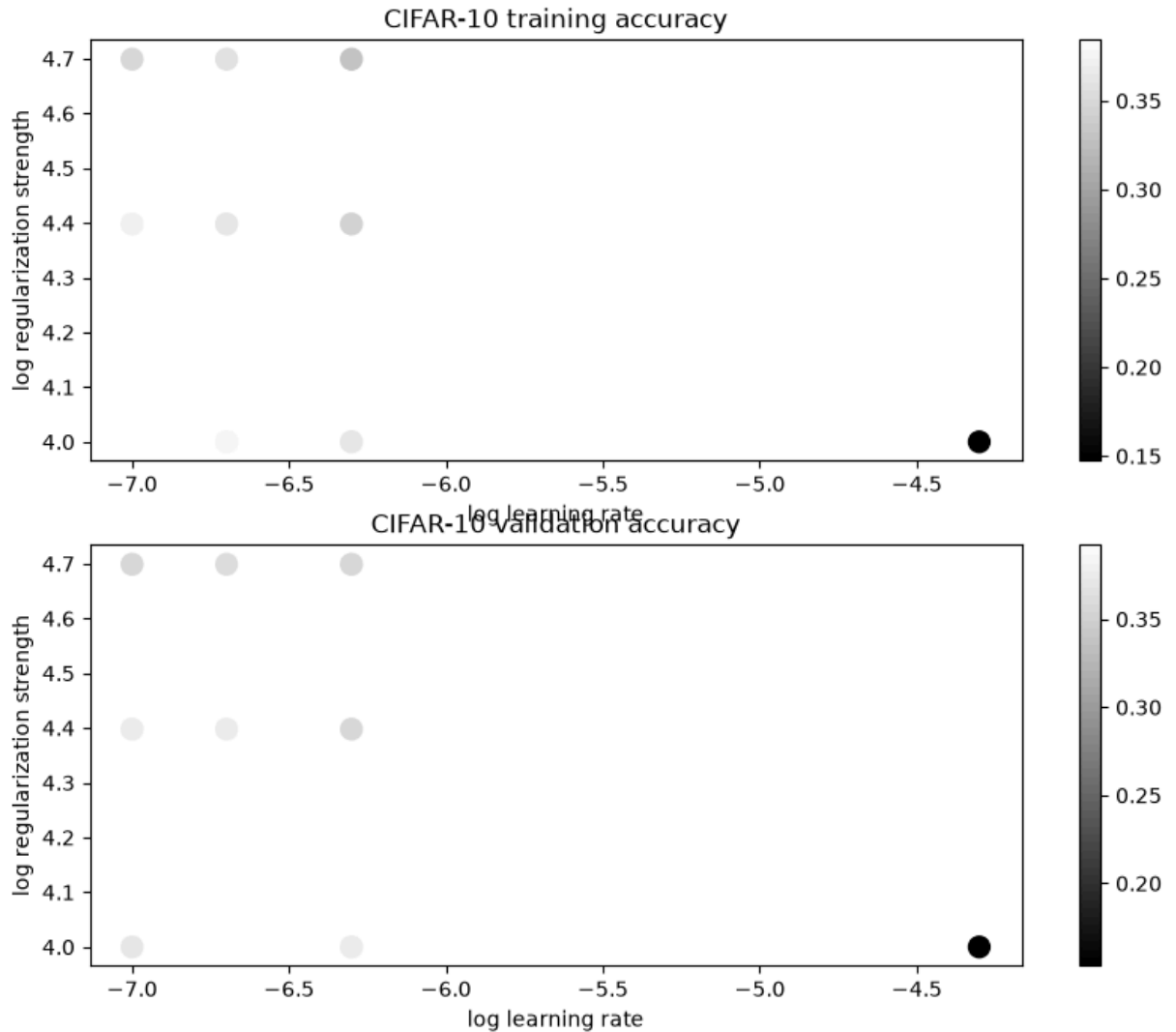
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.384163 val accuracy: 0.37
2000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.372245 val accuracy: 0.37
5000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.348755 val accuracy: 0.35
5000
lr 2.000000e-07 reg 1.000000e+04 train accuracy: 0.376918 val accuracy: 0.39
2000
lr 2.000000e-07 reg 2.500000e+04 train accuracy: 0.361939 val accuracy: 0.37
6000
lr 2.000000e-07 reg 5.000000e+04 train accuracy: 0.356857 val accuracy: 0.36
3000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.361286 val accuracy: 0.37
5000
lr 5.000000e-07 reg 2.500000e+04 train accuracy: 0.343776 val accuracy: 0.35
7000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.328265 val accuracy: 0.35
4000
lr 5.000000e-05 reg 1.000000e+04 train accuracy: 0.147224 val accuracy: 0.15
3000
best validation accuracy achieved during cross-validation: 0.392000
```

```
In [82]: # Visualize the cross-validation results
import math
x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
```

```
plt.title('CIFAR-10 validation accuracy')
plt.show()
```



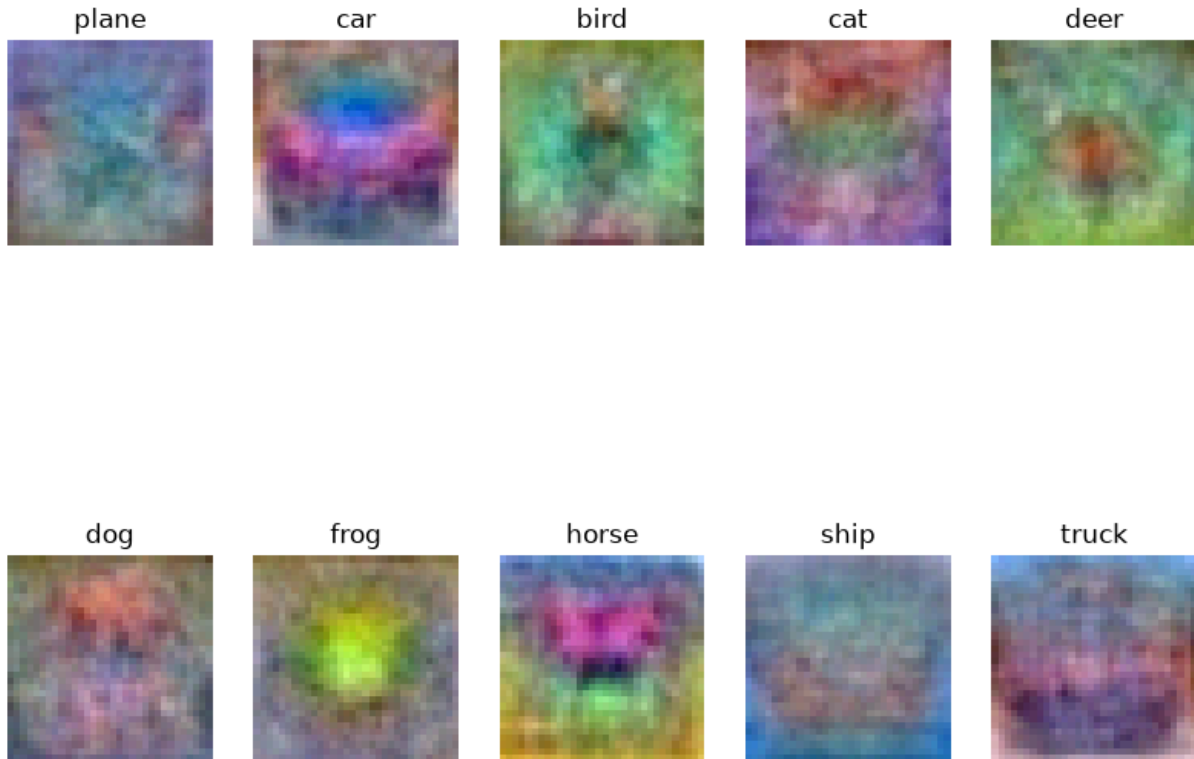
```
In [83]: # Evaluate the best svm on test set
y_test_pred = best_svm.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('linear SVM on raw pixels final test set accuracy: %f' % test_accuracy)
```

linear SVM on raw pixels final test set accuracy: 0.362000

```
In [84]: # Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, the
# or may not be nice to look at.
w = best_svm.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 's
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
```

```
plt.axis('off')
plt.title(classes[i])
```



Inline question 2:

Describe what your visualized SVM weights look like, and offer a brief explanation for why they look the way that they do.

Answer: The weights look like blurry, grainy templates of each class, dominated by color. The clearest ones: *car* has a red body with a blue area on top (windshield/sky), *frog* is a bright green blob in the center, *deer* has a brown center on a green background, *horse* has a dark and pinkish shape in the middle above a yellow-brown bottom, *truck* has a red/purple lower half, and *plane* and *ship* are bluish overall, with *ship* darker blue at the bottom (water). *Bird*, *cat* and *dog* are fainter reddish-brown or greenish blobs.

Why: A linear classifier $f(x, W) = Wx$ scores class j as $s_j = W_j \cdot x_i$, so each W_j acts as a template matched against the image. Training pushes W_j toward the pixel patterns common in class j , mostly overall color and background layout, which is why color blobs dominate. Each class gets only one template, so all poses, colors and left/right orientations in a class are averaged together, which makes the templates blurry. They stay grainy because the best model used the weakest regularization I tried ($\lambda = 1e4$, with $lr = 2e-7$) and only 1500 SGD iterations: weak L2 regularization $\lambda R(W) = \lambda \sum w^2$ does little to suppress weights on individual noisy pixels. Stronger regularization smooths the templates but lowered validation accuracy in my sweep (0.376 at $\lambda = 2.5e4$ and 0.363 at $\lambda = 5e4$, vs 0.392 at $1e4$, for $lr = 2e-7$)